

mcilloni's blog

Rants, experiments and experiences collected while delving into the magical world of computers.

[Home](#)

[About me](#)



© 2024. All rights reserved.

Bring your Arch Linux install everywhere

25 Oct 2023

The first time I installed Arch Linux was in 2007. In that foregone time, the only supported architecture was 32-bit x86, and the ISOs carried dubious release names such as “0.8 Voodoo”.

Despite the fact Arch shipped an installer that looked *suspiciously alike* FreeBSD's, you still had to configure a big deal of stuff by hand, using a slew of files with BSD-sounding names (`rc.conf` , anyone?). Xorg was the biggest PITA¹, but tools such as `xorgconfigure` and shoddy patched Xorg servers helped users achieve the “*Linux dream*”, which at the time mostly consisted of wobbly Beryl windows and spinny desktop cubes. That was the real deal back then, and nothing gave you more street cred than having windows that wobbled like cubes of jelly.

Those days are (somewhat sadly) long gone. Today's GNU/Linux distros are rather simple to install and setup, with often little to no configuration required (unless you are unlucky, of course). Distro targeted to advanced users, such as Arch, still require you to configure everything to your liking by yourself, but the overall stack (kernel, udev, Xorg, Wayland, ...) is now exceptionally good at automatically configuring itself based on the current hardware. UEFI also smoothes a lot of warts about the booting process.

This, alongside ultra-fast USB drive bays, makes self-configuring portable installs a concrete reality. I have now been using Arch Linux installs from SSDs in USB caddies for years now, for both work, system recovery and easy access to a ready-to-use environment from any computer. Despite the tradeoffs, it's remarkably solid and convenient.

In this post, I'll show step-by-step (with examples) how to install Arch Linux on a USB drive, and how to make it bootable everywhere², including virtual machines. I will try to cover as much corner cases as possible, but as always feel free to comment or contact me if you think something may be missing.

With a few adaptations, this guide may also be helpful to install Arch Linux on a non-mobile drive, if you so desire.

Prerequisites

- **An SSD drive in a USB enclosure.** In my experience, “*pre-made*” external USB disks are designed for storage and tend to perform way worse than internal drives in an external caddy. Enclosures also have the extra advantage improves durability³, and allows you to pop the SSD in a computer if you *really* need to.

The SSD can be either SATA or NVMe, with the latter being the *obviously* better choice. In general I tend to often run out of space on my machines, so I tend to use whatever SSD I have lying around. I honestly wouldn't bother with anything smaller than 128GB, unless you really plan to only use it for system recovery or light browsing.

NEVER use mechanical drives anymore or, at least, avoid them for anything that isn't cold storage or NAS. This is even more true for this project: not only spinning rust is *atrociously slow*, because most if not all 2.5" drives only spin at 5400 RPM, but they are also outrageously fragile and they almost always take more power than it's supplied by the average USB port.

- **An x86-64 computer.** It doesn't have to run Arch Linux, but it makes the whole process easier (especially if you wish to clone your current system to the USB drive - see below). A viable option is also to boot from the Arch Linux ISO and install the system from there, as long as you have another machine with a working internet connection to Google any issues you might encounter.

Note: I will only cover x86-64 with UEFI because that's by far the easiest and more reliable setup. BIOS requires tinkering with more complex bootloaders (such as SYSLINUX), while 32-bit x86 is not supported by Arch Linux anymore. ⁴

- **A working internet connection** - for obvious reasons. If you don't have an internet connection, then you probably have bigger problems to worry about than installing Arch Linux on a USB drive.

Setting up the drive

Given that we are talking about a portable install, disk encryption is nothing short of mandatory. In general, I think that encrypting your system is ALWAYS a good idea, even if you don't plan to carry it around much ⁵.

The choices of filesystem and encryption scheme are up to you, but there are basically three options I've used and I can recommend:

1. **LUKS with a classic filesystem**, such as `ext4`, `F2FS` or `XFS`. This is the simplest option, and it is probably more than enough for most people.
2. **ZFS with native encryption**. I must admit, this *may* be somewhat overkill, but it's also my favourite because due to it being such a great experience overall. While ZFS isn't probably the best choice for a removable hard drive, it's outstandingly solid, supports compression, snapshots and checksumming - all things I do want from a system that runs from what's potentially a flimsy USB cable. ⁶ I am yet to lose any data due to ZFS itself, and I have been using it for the best part of a decade now.

ZFS is also what I use on all my installs, so I can easily migrate datasets from/to a USB-installed system using the `zfs send` / `zfs receive` commands if I need to, or quickly back up the whole system.

Native ZFS encryption, while not as thoroughly tested and secure as `LUKS`, is still probably fine for most people, while also ridiculously convenient to set up. If that's not enough for you, using `ZFS` on top of `LUKS` is still an acceptable choice (albeit more complicated to pull off).

3. **LUKS with BTRFS**. I have also used this setup in the past, and there's a lot to like about it, such as the fact that BTRFS supports lots of ZFS's best features without requiring to install any out-of-tree kernel modules - a very nice plus indeed.

Sadly, I have been burnt by BTRFS so many times in the past 12 years that I can't honestly say I would want to entrust it with my data any time soon. YMMV, so maybe give it a try if you're curious.

Regardless of that, I will now cover all three options in the next sections.

One important note: I *deliberately* decided to leave kernel images unencrypted (in UKI form) in the ESP, sticking with full encryption for just the root filesystem. My main concern is about protecting the data stored on the drive in case it's lost or broken, and I assume nobody will attempt evil maid attacks.⁷ Encrypting the kernel is also probably rather pointless without a signed bootloader and kernel - something that's very hard to setup for a portable USB setup.

I also will not show how to set-up UEFI Secure Boot. While having Secure Boot enabled is a good thing in general, it makes setting the system up vastly more complex, for debatable benefits. This setup is in general not meant to be used for security critical systems, but to provide a convenient way to carry a working environment around between machines you have complete control of.

0. (Optional) Obtaining a viable ZFS setup environment

Unfortunately, ZFS on Linux is an out-of-tree filesystem. This basically means that it's not bundled with the kernel as with all other filesystem, but instead it's distributed by an independent project and has to be compiled and installed separately. This is due to a complex licensing incompatibility between the **CDDL** license used by OpenZFS, and the **GPLv2** license used by the Linux kernel, which makes it impossible to ever bundle ZFS and Linux together.

If you intend on using ZFS, you must follow these steps first; if not, just skip to the section 1.

This procedure varies depending on the distribution you are using:

0.1. Arch Linux

Arch doesn't distribute ZFS due to the aforementioned licensing issues, but it's readily available and readily maintained by the [ArchZFS project](#), both in form of AUR `PKGBUILD`s and in the third party repository `archzfs`.

The packages you are going to need are `zfs-utils` and a module compatible with your current kernel; the latter can either come from a kernel-specific package (i.e. `zfs-linux-lts`), or a DKMS one (i.e. `zfs-dkms`).

If you opt to install the packages from ArchZFS, add the `[archzfs]` repository to your `pacman.conf` (look at [Arch Wiki](#) for the correct URL), remembering to import the PGP key using `pacman-key -r KEY` followed by `pacman-key --lsign-key KEY`.

If you need to boot from an ISO, it's a bit more complicated, so I won't specify the details here because it would be quite long. Give a look at [this repo](#) for a quick way to generate one. If you feel adventurous, you can also try to use an ISO with ZFS support from another distribution (such as Ubuntu) and follow the instructions below to set up a working environment.

0.2. Other Linux distributions

If you are starting from another distribution, you will need to visit the [OpenZFS on Linux](#) website and follow the instructions for your distribution (if included).

This will generally involve adding a third party repository (except for Ubuntu, which has ZFS in its main repos), and following the instructions.

For instance, on Debian it's recommended to enable the `backports` repository, in order to install a more up to date version. This also requires to modify APT's settings by pinning the `backports` repository to a higher priority for the ZFS packages.

```
# cat <<'EOF' > /etc/apt/sources.list.d/bookworm-backports.list
deb http://deb.debian.org/debian bookworm-backports main contrib
deb-src http://deb.debian.org/debian bookworm-backports main contrib
EOF
# cat <<'EOF' > /etc/apt/preferences.d/90_zfs
```

```
Package: src:zfs-linux
Pin: release n=bookworm-backports
Pin-Priority: 990
# apt update
# apt install dpkg-dev linux-headers-generic linux-image-generic
# apt install zfs-dkms zfsutils-linux
[...]
```

Regardless of what you are using, you should now have a working ZFS setup. You can verify this by running `zpool status` ; if it prints `no pools available` instead of complaining about missing kernel modules, you are good to go, and you may start setting up the drive.

1. Partitioning

From either the Arch Linux ISO or your existing system, run a disk partitioning tool. I'm personally partial to `gdisk` , but `parted` and `fdisk` are also fine ⁸. `parted` also has a graphical frontend, `gparted` , which is very easy to use, in case you are afraid to mess up the partitioning and prefer having clear feedback on what you're doing ⁹.

The partitioning scheme is generally up to you, with the bare minimum being:

- A **FAT32 EFI System Partition** (ESP), possibly with a comfortable size of at least 300 MiB. This is where the UKI (and optionally, a bootloader) will be stored. I do not recommend going for BIOS/MBR, given that x64 computers have supported UEFI for more than a decade now.

The ESP will be mounted at `/boot/efi` in the final system.

- A **root partition**. This is where the system will be installed and all files will be stored. The size is up to you, but I would recommend at least 20 GiB for a *very* minimal system. While the system itself doesn't necessarily need a lot of space, with limited storage space you will find yourself often cleaning up the package caches, logs and temporary files that litter `/var` .

The root partition will also be our encryption root, and it will be formatted with either `LUKS` or `ZFS` .

While some guides may suggest also creating a swap partition, I generally don't recommend using one when booting from USB,. Swapping to storage will quickly turn into a massive bottleneck and slow down the whole system to a crawl. If you really need swap, I would recommend looking into alternatives such as `zram` or `zswap` , which are probably a wiser choice.

Also, it goes without saying, do not hibernate a system that runs from a USB drive, unless you plan on resuming it on the same machine.

1.1 Creating the partition label

Feel free to skip to the next step if you already have a partition label on your drive, with two sufficiently sized partitions for the ESP and the root filesystem, and you don't want to use the whole drive.

First, identify the device name of your USB drive. In my case, it's a Samsung 960 EVO 250 GB NVMe drive inside a USB 3.2 enclosure:

```
id/usb*
 9 Aug 10 18:42 /dev/disk/by-id/usb-Samsung_SSD_960_EVO_250G_012938001243-0:0 -> ../../sdb
10 Aug 10 18:42 /dev/disk/by-id/usb-Samsung_SSD_960_EVO_250G_012938001243-0:0-part1 -> ../../sdb1
10 Aug 10 18:42 /dev/disk/by-id/usb-Samsung_SSD_960_EVO_250G_012938001243-0:0-part2 -> ../../sdb2
```

I can't stress this enough: **DO NOT USE RAW DEVICE NAMES WHEN PARTITIONING!** Always use `/dev/disk` when performing destructive operations of block devices - it's not a matter of *if* you will lose data, but *when*.¹⁰
`/dev/disk/by-id` is by far the best good choice due to how it clearly names devices by bus type, which makes very hard to mix up devices by mistake.

Once you have identified the device name, run `gdisk` (or whatever you prefer) and create a new GPT label in order to wipe the existing partition table.

```
# gdisk /dev/disk/by-id/usb-Samsung_SSD_960_EVO_250G_012938001243-0:0
GPT fdisk (gdisk) version 1.0.9.1

Partition table scan:
  MBR: not present
  BSD: not present
  APM: not present
  GPT: not present

Creating new GPT entries in memory.

Command (? for help): o
This option deletes all partitions and creates a new protective MBR.
Proceed? (Y/N): Y

Command (? for help): p
Disk /dev/disk/by-id/usb-Samsung_SSD_960_EVO_250G_012938001243-0:0: 488397168 sectors, 232.9 GiB
Sector size (logical/physical): 512/512 bytes
Disk identifier (GUID): 55F7C0C7-35B3-44C5-A2C4-790FE33014FD
Partition table holds up to 128 entries
Main partition table begins at sector 2 and ends at sector 33
First usable sector is 34, last usable sector is 488397134
Partitions will be aligned on 2048-sector boundaries
Total free space is 488397101 sectors (232.9 GiB)

Number   Start (sector)    End (sector)  Size      Code  Name

Command (? for help):
```

1.2. Creating the EFI System Partition (ESP)

With now a clear slate, we can create an EFI partition (GUID type `EF00`). The ESP not be encrypted and will contain the Unified Kernel Image the system will boot from; for this reason, I recommend giving it at least 300 MiB of space in order to avoid unpleasant surprises when updating the kernel.

```
Command (? for help): n
Partition number (1-128, default 1): 1
First sector (34-488397134, default = 2048) or {+}size{KMGTP}:
Last sector (2048-488397134, default = 488396799) or {+}size{KMGTP}: +300M
Current type is 8300 (Linux filesystem)
Hex code or GUID (L to show codes, Enter = 8300): ef00
Changed type of partition to 'EFI system partition'
```

Notice how I left the first sector blank, and I specified `+300M` as the last sector. This is because I want `gdisk` to automatically align the partition to the nearest sector boundary (2048 in this case). `gdisk` tends to be quite good at automatically deducing the correct alignment, a process that tends to be finicky with USB enclosures.

I also highly recommend giving the partition a GPT name (which will be visible under `/dev/disk/by-partlabel`):

```
Command (? for help): c
Using 1
Enter name: ExtESP
```

```
Command (? for help):
```

1.3. Creating the root partition

Finally, we can create the root partition. This partition will be encrypted, and will contain the system and all of user data. I recommend giving it at least 20 GiB of space, but feel free to use more if you have some spare room.

For instance, the following command will create a partition using all of the remaining space on the drive:

```
Command (? for help): n
Partition number (2-128, default 2): 2
First sector (34-488397134, default = 616448) or {+}size{KMGTP}:
Last sector (616448-488397134, default = 488396799) or {+}size{KMGTP}:
Current type is 8300 (Linux filesystem)
Hex code or GUID (L to show codes, Enter = 8300):
Changed type of partition to 'Linux filesystem'

Command (? for help): c
Partition number (1-2): 2
Enter name: ExtRoot
```

Feel free to leave use the default GUID type (8300) for the root partition, as it will be changed when formatting the partition later.

1.4. Writing the partition table

Once you are done, you should have a partition table resembling the following:

```
Command (? for help): p
Disk /dev/disk/by-id/usb-Samsung_SSD_960_EVO_250G_012938001243-0:0: 488397168 sectors, 232.9 GiB
Sector size (logical/physical): 512/512 bytes
Disk identifier (GUID): AABE7D47-3477-4AB6-A7C1-BC66F87CB1C1
Partition table holds up to 128 entries
Main partition table begins at sector 2 and ends at sector 33
First usable sector is 34, last usable sector is 488397134
Partitions will be aligned on 2048-sector boundaries
Total free space is 2349 sectors (1.1 MiB)

Number  Start (sector)    End (sector)  Size      Code  Name
   1            2048             616447    300.0 MiB   EF00   ExtESP
   2          616448         488396799    232.6 GiB   8300   ExtRoot
```

If everything looks OK, proceed to commit the partition table to disk. Again, ensure that you are writing to the correct device, that it does not contain any important data, and no old partition is mounted:

```
Command (? for help): w

Final checks complete. About to write GPT data. THIS WILL OVERWRITE EXISTING
PARTITIONS!!

Do you want to proceed? (Y/N): Y
OK; writing new GUID partition table (GPT) to /dev/disk/by-id/usb-Samsung_SSD_960_EVO_250G_012938001243-
The operation has completed successfully.
```

2. Creating the filesystems

Now that we created a viable partition layout, we can proceed by creating filesystems on it.

As I've mentioned before, there are several potential choices regarding what filesystems and encryption schemes to use. Regardless of what you'll end up choosing, the ESP must always be formatted as FAT (either FAT32 or FAT16):

```
# mkfs.fat -F 32 -n EXTESP /dev/disk/by-id/usb-Samsung_SSD_960_EVO_250G_012938001243-0:0-part1
mkfs.fat 4.2 (2021-01-31)
```

After doing this, proceed depending on what filesystem you want to use.

2.1. Straightforward: LUKS with a native filesystem

LUKS with a simple filesystem is by far the simplest solution, and (probably) the “safest” for what regards setup complexity. LUKS can also be used with LVM2 for more “advanced” solutions, but it goes beyond the scope of this post.

As I've mentioned previously, we are going to set up full encryption for system and user data, but not for the kernel, which will reside in UKI form inside the ESP. If you are interested in a more “paranoid” setup, you can find more information in the [Arch Wiki](#).

2.1.1. Creating the LUKS container

First, we need to format the previously created partition as a LUKS container, picking a good passphrase in the process. What makes a good passphrase [is a whole topic in itself](#), and recommendations tend to change frequently following the current trends and cracking techniques. Personally, I recommend using a passphrase that is easy to remember but computationally hard to guess for a computer, such as a (very) long password full of spaces, letters, numbers and special characters.

```
# cryptsetup luksFormat --label ExtLUKS /dev/disk/by-id/usb-Samsung_SSD_960_EVO_250G_012938001243-
WARNING!
=====
This will overwrite data on /dev/disk/by-id/usb-Samsung_SSD_960_EVO_250G_012938001243-0:0-part2 ir

Are you sure? (Type 'yes' in capital letters): YES
Enter passphrase for /dev/disk/by-id/usb-Samsung_SSD_960_EVO_250G_012938001243-0:0-part2:
Verify passphrase:
Ignoring bogus optimal-io size for data device (33553920 bytes).
```

Note that I deliberately stuck with the default settings, which are good enough for most use cases.

After creating the container, we need to open it in order to format it with a filesystem:

```
# cryptsetup open /dev/disk/by-id/usb-Samsung_SSD_960_EVO_250G_012938001243-0:0-part2 ExtLUKS
Enter passphrase for /dev/disk/by-id/usb-Samsung_SSD_960_EVO_250G_012938001243-0:0-part2:
```

`ExtLUKS` is an arbitrary name I chose for the container - feel free to pick whatever name you like. Whatever your choice is, after successfully unlocking the LUKS container it will be available as a block device under `/dev/mapper/<name>` :

```
$ ls -ll /dev/mapper/ExtLUKS
lrwxrwxrwx 1 root root 7 Aug 28 22:45 /dev/mapper/ExtLUKS -> ../dm-0
```

2.1.2. Formatting the container

Now that we have an unlocked LUKS container, we can format it with a “real” filesystem. Note that, if you wish to use LVM2, this would be the right time to create the LVM volumes.

No matter the filesystem you plan to use over LUKS, *ext4*, *F2FS*, *XFS* and *Btrfs* are all created via the respective `mkfs` tool:

```
# mke2fs -t ext4 -L ExtRoot /dev/mapper/ExtLUKS # for Ext4
# mkfs.f2fs -l ExtRoot /dev/mapper/ExtLUKS # for F2FS
# mkfs.xfs -L ExtRoot /dev/mapper/ExtLUKS # for XFS
# mkfs.btrfs -L ExtRoot /dev/mapper/ExtLUKS # for Btrfs
```

2.2. Advanced: Btrfs subvolumes

If you picked a “plain” filesystem such as *ext4*, *F2FS* or *XFS*, you can skip this section.

In case you picked *Btrfs*, it's a good idea to create subvolumes for `/` and `/home` in order to take advantage of Btrfs's snapshotting capabilities.

Compared to older filesystems, Btrfs and ZFS have the built-in capability to create logical subvolumes (*datasets* in ZFS parlance) that can be mounted, snapshotted and managed independently. This is somewhat similar to LVM2, but immensely more powerful and flexible; all subvolumes share the same storage pool and can have different properties enabled (such as compression or CoW), or ad-hoc quotas and mount options.

Compared to other filesystems, Btrfs (and ZFS) requires filesystems to be online and mounted in order to perform operation on them, such as scrubbing (an operation akin to `fsck`) and subvolume management.

2.2.1. Mounting the root subvolume

Mount the filesystem on a temporary mountpoint:

```
# mount /dev/mapper/ExtLUKS /path/to/temp/mount
# mount | grep ExtLUKS
/dev/mapper/ExtLUKS on /path/to/temp/mount type btrfs (rw,relatime,ssd,space_cache=v2,subvolid=5,s
```

Notice how `mtab` includes the options `subvolid=5,subvol=/'`. This means that the *default subvolume* has been mounted, identified with the ID `5` and named `/'`. This is the subvolume that will be mounted by default, acting as the root parent of all other subvolumes.

2.2.2. Creating the subvolumes

Now we can create the subvolumes for `/` and `/home`, called `@` and `@home` respectively:

```
# btrfs subvolume create /path/to/temp/mount/@ # for /
Created subvolume '/path/to/temp/mount/@'
# btrfs subvolume create /path/to/temp/mount/@home # for /home
Created subvolume '/path/to/temp/mount/@home'
```

Using a `@` prefix with Btrfs subvolumes is long established convention. The situation should now look like this:

```
# btrfs subvolume list -p /path/to/temp/mount
ID 256 gen 8 parent 5 top level 5 path @
ID 257 gen 9 parent 5 top level 5 path @home
# ls -ll /path/to/temp/mount
@
@home
```

Notice how in *Btrfs* subvolumes *are also subdirectories* of their parent subvolume. This is very useful when mounting the disk as an external drive. Subvolumes can also be mounted directly by passing the `subvol` and


```
subvolid to mount .
```

Before moving to the next step, remember to unmount the root subvolume.

2.3. Advanced: ZFS with native encryption

My personal favourite, ZFS is a rocksolid system that's ubiquitous in data storage, thanks to its impressive stability record and advanced features such as deduplication, built-in RAID, ...

Albeit arguably less flexible than Btrfs, which was originally designed as a Linux-oriented replacement for the CDDL-encumbered ZFS, in my experience ZFS tends to be vastly more stable and reliable in day to day use. In the last 6 years, I have almost exclusively used ZFS on all my computers, and I have yet to lose any data due to ZFS itself. ¹¹

ZFS is quite different compared to other filesystems. Instead of filesystems, ZFS works on *pools*, which consists in collections of one or more block devices (potentially in *RAID* configurations). Every pool can be divided into a hierarchy of *datasets*, which are roughly equivalent to subvolumes in Btrfs.

Datasets can be mounted independently, and can each have their own properties, such as compression, quotas, and so on, which may either be set per-dataset or inherited from the parent dataset.

Compared to Btrfs, ZFS manages its own mountpoints as inherent properties of the dataset. This is both incredibly useful and bothersome; on one hand, having mountpoints intrinsically related to datasets allows for easier management and more clarity than legacy mounting, but on the other hand it may turn confusing and inflexible when managing complex setups. In any case, you can opt-out from letting ZFS managing mountpoints for a given dataset by setting the mountpoint to `legacy`, and mounting it manually as you would with any other filesystem.

2.3.1. Creating the ZFS pool

Our case is quite simple, given that we only have a single drive.

Create a new dataset called `extzfs` (or whatever you prefer), being careful to specify an `altroot` via `-R`. Otherwise, the new mountpoints will override your system ones as soon as you set up the pool:

```
# zpool create -m none -R /tmp/mnt extzfs /dev/disk/by-id/usb-Samsung_SSD_960_EVO_250G_01293800124
```

You may have to specify `-f` if the partition wasn't empty before. Note the `-m none` option, which will set no mountpoint for the root dataset of the pool itself. Compared to Btrfs, ZFS doesn't expose datasets as subdirectories of their parent pool, so it makes little sense to allow mounting the root dataset.

2.3.2. Creating an encrypted dataset root

As mentioned before, we are going to use native ZFS encryption, which is generally considered safe, but it *may* not be as water-tight and battle-tested as LUKS; this is generally not a problem for most people except the most paranoid. If you count yourself among their ranks, remember that you can always use LUKS on top of ZFS. It may end up being more complex, but it's a viable option.

First, we need to create an encrypted dataset; this will act as the encryption root for all the other datasets. We will (arbitrarily) call it `extzfs/encr`:

```
# zfs create -o encryption=on -o keyformat=passphrase -o keylocation=prompt -o mountpoint=none -o
Enter new passphrase:
Re-enter new passphrase:
```

Notice that we are using the `passphrase` key format alongside the `prompt` key location. This means that ZFS will expect the encryption key in the form of a password entered by the user. Another option would be to use a key file, which is arguably more secure but also incredibly more cumbersome to use for the root device, so I'll leave how to use one as an exercise to the reader.

Like with LUKS, I recommend picking a safe password that's easy to remember but hard to guess. See paragraph 2.2.1. for more details.

Also, like in Btrfs's case I will enable compression in order to spare some space on my small SSD. This *may* potentially leak a bit of information about the data contained inside the encrypted container, but it's generally not a problem for most people.

2.3.3. Creating the system dataset

Now that we have an `encryptionroot`, we can create all the datasets we need under it, and they will be encrypted and unlocked automatically along with it.

Keeping in mind that's good practice to create a hierarchy that allows for the quick and easy creation of new boot environments, under `encr` we are going to create:

1. A `root` dataset, which will not be mounted, and under which we will place datasets contain system images
2. A `home` dataset, which will act as the root for all user-data datasets
3. A `default` dataset under `root`, which will be mounted as `/` and contain the system we're going to install
4. A `logs` dataset for `/var/log` under `default`, which is required to be a separate dataset in order to enable the ACLs required by `systemd-journald`
5. `users` and `root` datasets under `home`, which will respectively be mounted as `/home` and `/root`.

```
# zfs create -o mountpoint=none extzfs/encr/root
# zfs create -o mountpoint=none extzfs/encr/home
# zfs create -o mountpoint=/ extzfs/encr/root/default
# zfs create -o mountpoint=/var/log -o acltype=posixacl extzfs/encr/root/logs
# zfs create -o mountpoint=/home extzfs/encr/home/users
# zfs create -o mountpoint=/root extzfs/encr/home/root
```

After running the commands above, the situation should look like the following:

```
# zfs list
```

NAME	USED	AVAIL	REFER	MOUNTPOINT
extzfs	1.20M	225G	24K	none
extzfs/encr	721K	225G	98K	none
extzfs/encr/home	294K	225G	98K	none
extzfs/encr/home/root	98K	225G	98K	/tmp/mnt/root
extzfs/encr/home/users	98K	225G	98K	/tmp/mnt/home
extzfs/encr/root	329K	225G	98K	none
extzfs/encr/root/default	231K	225G	133K	/tmp/mnt
extzfs/encr/root/logs	98K	225G	98K	/tmp/mnt/var/log

Notice how all mountpoints are relative to `/tmp/mnt`, which is the alternate root the `extzfs` pool was imported with (in this case, created) using the `-R` flag. The prefix will be stripped when importing the pool on the final system, leaving only the real mountpoints. This feature makes mounting systems installed on ZFS incredibly convenient, because the entire hierarchy is properly mounted under any directory you choose, allowing to rapidly chroot into the system and perform emergency maintenance operations.

2.3.4. Setting the bootfs

The pool's `bootfs` property can be used to indicate which dataset contains the desired boot environment. This is not necessary, but it helps simplifying the kernel command line.

Run the following command to set the `bootfs` property to `extzfs/encr/root/default` :

```
# zpool set bootfs=extzfs/encr/root/default extzfs
```

For the sake of consistency, export now the pool before moving to the next step. This is not strictly necessary, but it doesn't hurt to ensure that the pool can be correctly imported using the given passphrase.

To export the pool and unmount all datasets, run:

```
# zpool export extzfs
```

3. Installing Arch Linux

Installing Arch Linux is not the complex task it was a few decades ago. Arguably, it requires a bit of knowledge and experience, but it's not out of reach for most tech-savvy users.

In general, when installing Arch onto a new drive (in this case, our portable SSD), there are two basic approaches:

1. Install a fresh system from either an existing Linux install¹² or the Arch Linux ISO;
2. Clone an existing system to the new drive.

I'll go cover both approaches in the next sections, alongside with a few tips and tricks I've learnt over the years.

3.1. Mounting the filesystems

Regardless on the filesystem or approach you've picked, you should now mount the root filesystem on a temporary mountpoint. I will use `/tmp/mnt` , but feel free to use whatever you prefer:

```
# mkdir /tmp/mnt
```

If using LUKS:

```
# cryptsetup open /dev/disk/by-id/usb-Samsung_SSD_960_EVO_250G_012938001243-0:0-part2 ExtLUKS
```

and then, depending on the filesystem:

```
# mount /dev/mapper/ExtLUKS /tmp/mnt # for ext4/XFS/F2FS
```

```
# mount -o subvol=@,compress=lzo /dev/mapper/ExtLUKS, /tmp/mnt # for Btrfs
```

I've also enabled compression for Btrfs, which may or may not be a good idea depending on your use case. Notice that compressing data before encrypting it *may* hypothetically leak some info about the data contained. Avoid compression if you are concerned about this and/or you have a very large SSD.

If using ZFS, run:

```
# zpool import -l -d /dev/disk/by-id -R /tmp/mnt extzfs
```

and it should do the trick.

3.2. Installing from scratch

If in doubt, just refer follow the official [Arch Linux installation guide](#) - it will not cover all the details for advanced installs, but it's a good starting point.

In general, the steps somewhat resemble the following, regardless of what filesystem you've picked:

```
# mkdir -p /tmp/mnt/boot/efi # we still need to mount /boot, which is on a separate partition
# mount /dev/disk/by-id/usb-Samsung_SSD_960_EVO_250G_012938001243-0:0-part1 /tmp/mnt/boot/efi
```

3.2.1. Installing from an existing Arch Linux install

If you are running from an existing Arch Linux install or the Arch ISO, installing a base system is as easy as running `pacstrap` (from the `arch-install-scripts` package) on the mountpoint of the root filesystem:

```
# pacstrap -K /tmp/mnt base perl neovim
[lots of output]
```

I've also thrown in `neovim` because there are no editors installed by default in `base`, but feel free to use whatever you like. `perl` is also (implicitly) required by several packages, and not installing it may trigger unpredictable issues later.

Now enter the new system with `arch-chroot`:

```
# arch-chroot /tmp/mnt
```

3.2.2. Installing from a non-Arch system

All the steps above (except for `pacstrap`) can be performed from basically any Linux distribution. If you are running from a non-Arch system, don't worry - there are workarounds available for that.

An always viable solution is always [to use the bootstrap tarball from an Arch mirror](#).

A trickier (but arguably more fun) path is to build `pacman` from source, and then using it to install the base system. For instance, on Debian:

```
$ sudo apt install build-essential meson cmake libcurl4-openssl-dev libgpgme-dev libssl-dev libarc
[...]
$ wget -O - https://sources.archlinux.org/other/pacman/pacman-6.0.2.tar.xz | tar xvfJ -
$ cd pacman-6.0.2
$ meson setup --default-library static build # avoid linking pacman with the newly built shared li
[...]
$ ninja -C build
[...]
```

You should have a working `pacman` binary in `build/pacman`. In order to install the base system, you need to create a minimal `pacman.conf` file:

```
$ cat <<'EOF' >> build/pacman.conf
SigLevel = Never

[core]
Server = https://geo.mirror.pkgbuild.com/$repo/os/$arch
```

```
[extra]
Server = https://geo.mirror.pkgbuild.com/$repo/os/$arch
EOF
```

For this time only, I have disabled signature verification because going through the whole ordeal of setting up `pacman-key` and importing the Arch Linux signing keys for a makeshift `pacman` install is very troublesome. If you are *really* concerned about security, use the bootstrap tarball instead.

Create the required database directory for `pacman`, and install the same packages as above:

```
$ sudo mkdir -p /tmp/mnt/var/lib/pacman/
$ sudo build/pacman -r /tmp/mnt --config=build/pacman.conf -Sy base perl neovim
```

This will result in a working Arch Linux chroot, albeit only partially set up.

Chroot into the new system, and properly set up the Arch Linux keyring:

```
$ sudo mount --make-rslave --rbind /dev /tmp/mnt/dev
$ sudo mount --make-rslave --rbind /sys /tmp/mnt/sys
$ sudo mount --make-rslave --rbind /run /tmp/mnt/run
$ sudo mount -t proc /proc /tmp/mnt/proc
$ sudo cp -L /etc/resolv.conf /tmp/mnt/etc/resolv.conf
$ sudo chroot /tmp/mnt /bin/bash
[root@chroot /]# pacman-key --init
[root@chroot /]# pacman-key --populate archlinux
```

You can now proceed as if you were installing from an existing Arch Linux system.

3.2.3. Installing a kernel

In order to install packages inside your chroot, you need to enable at least one Pacman mirror first in `/etc/pacman.d/mirrorlist`. If you used `pacstrap` from an existing Arch Linux system, this may be unnecessary.

After enabling one or more mirrors, you can install a kernel of your choice:

```
[root@chroot /]# pacman -Sy linux-lts linux-lts-headers linux-firmware
```

Notice that I've chosen to install the LTS kernel, which is in general a good idea when depending on out-of-tree kernel modules such as ZFS or NVIDIA drivers. Feel free to install the latest kernel if you prefer, but remember to be careful when updating the system due to potential module breakage.

The command above will also generate an `initrd`, which we don't really need (we will use UKI instead). We will have to delete that later.

3.2.4. Installing the correct helpers for your filesystem

In order for `fsck` to properly run, or to mount ZFS, you need to install the correct package for your filesystem:

1. If you've installed your system over ZFS, this is a good time to set-up the ArchZFS repository in the chroot (see above)
2. If you've installed your system over Btrfs, you need to install `btrfs-progs`. `cryptsetup` should already have been pulled in as a dependency to `systemd`
3. If you are using another filesystem, install the correct package:
 - a. For `ext4`, `e2fsprogs` should already have been pulled in by dependencies installed by `base` - ensure you can run `e2fsck` from the chroot.

b. For `XFS` , install `xfspggs` .

c. For `F2FS` , install `f2fs-tools` .

Remember to also always install `dosfstools` , which is required to `fsck` the FAT filesystem on the ESP.

3.3. Cloning an existing system

Instead of installing the system from scratch, you may clone an existing system instead. Just remember after the move to

1. fix `/etc/fstab` with the new `PARTUUID` s
2. give the system an unique configuration (i.e., change the hostname, fix the hostid, ...) in order to avoid clashes
3. do not transfer the contents of the ESP - if you use UKI and mount it at `/boot/efi` , you will regenerate its contents later when you reapply the steps from above.

There are 3 feasible ways to do this.

3.3.1. Use `dd` to clone a partition block by block.

This methods has a few advantages, and quite a bit of downsides:

- **PRO:** because it literally clones an entire disk, byte per byte, to another, it is the most conservative method among all
- **CON:** because it clones an entire disk byte per byte, issues such as fragmentation and data in unallocated sectors are copied
- **CON:** because it clones an entire disk byte per byte, the target partition or disk must be at least as large as the source, or the source must be shrunk beforehand, which is not always possible (like with XFS)

If you opt for this solution, just run `dd` and copy one or more existing partitions to the LUKS container:

```
# dd if=/path/to/source/partition of=/dev/mapper/ExtLUKS bs=1M status=progress
```

3.3.2. Use `rsync` to clone a filesystem onto a new partition.

This method is the most flexible, because it's completely agnostic regarding the source and destination filesystems, as long as the destination can fit all contents from the source. Just mount everything where it's supposed to go, and run (as root):

```
# rsync -qaHAXS /{bin,boot,etc,home,lib,opt,root,sbin,usr,var} /tmp/mnt/dest
```

The root has now been cloned, but it's missing some base directories.

Given that I assume we are booting from an Arch Linux system, just reinstall `filesystem` inside the new root:

```
$ sudo pacman -r /tmp/mnt --config /tmp/mnt/etc/pacman.conf -S filesystem
```

This will fixup any missing directory and symlink, such as `/dev` , `/proc` , ... Notice that only for this time I have used the `-r` parameter. This changes pacman's root directory, and should always used with extreme care.

3.3.3. Use Btrfs snapshotting and replication facilities to clone existing subvolumes.

Btrfs supports incremental snapshotting and sending/receiving them as incremental data streams. This is extremely convenient, because replication ensures that files are transferred perfectly (with the right permissions, metadata, ...) without having to copy any unnecessary empty space.

In order to duplicate a system using Btrfs, partition and format the disk as described above, and then snapshot and send the subvolumes to the new disk. Assuming the root subvolume has been mounted under `/tmp/src`

```
# mount -o subvol=/ /path/to/root/dev /tmp/src
# mount -o subvol=/ /dev/mapper/ExtLUKS /tmp/mnt
# btrfs su snapshot -r /tmp/src/@{-mig}
Create a readonly snapshot of '/tmp/src/@' in '/tmp/src/@-mig'
# btrfs su snapshot -r /tmp/src/@home{-mig}
Create a readonly snapshot of '/tmp/src/@home' in '/tmp/src/@home-mig'
# btrfs send /tmp/src/@-mig | btrfs receive /tmp/mnt
At subvol /tmp/src/@-mig
At subvol @-mig
# btrfs send /tmp/src/@home-mig | btrfs receive /tmp/mnt
At subvol /tmp/src/@home-mig
At subvol @home-mig
```

The system has now been correctly transferred. Rename the subvolumes to their original names and delete the now unnecessary snapshots if you want to reclaim the space ¹³:

```
# perl-rename -v 's/\-mig//g' /tmp/mnt/@*
/tmp/mnt/@-mig -> /tmp/mnt/@
/tmp/mnt/@home-mig -> /tmp/mnt/@home
# btrfs su delete /tmp/src/@*-mig
Delete subvolume (no-commit): '/tmp/src/@-mig'
Delete subvolume (no-commit): '/tmp/src/@home-mig'
# umount /tmp/{src,mnt}
# mount -o subvol=@,compress=lzo /dev/mapper/ExtLUKS /tmp/mnt
```

Unmount the root subvolume and mount the system as you normally would. You are now ready to move to the next step.

3.3.4. Use ZFS snapshotting and replication facilities to clone existing datasets.

With ZFS, the process is very similar to Btrfs, with a few different steps depending if your source datasets are already encrypted or not.

After creating a pool, snapshot your root disk *recursively*. If your system resides on an encrypted dataset, snapshotting the encryption root will also snapshot all the datasets contained within it:

```
# zfs snapshot -r zroot/encr@migration # otherwise, snapshot all the required datasets
```

After doing that, you can either:

1. create a new encrypted dataset and send the unencrypted snashots to it:

```
# zfs create -o encryption=on -o keyformat=passphrase -o keylocation=prompt -o mountpoint=nc
# for DATASET in root home ... # note: replace with the actual datasets
do zfs send zroot/$DATASET@migration | zfs recv extzfs/encr/$DATASET
# ...
```

Migrating unencrypted datasets to an encrypted root dataset requires transferring the snapshots one by one. It's generally easier to just let the newly received snapshots inherit properties from their parents, and then fixing mountpoints and other properties later using `zfs set`. You can also do it directly, if necessary, by setting the properties using the `-o` flag with `zfs recv`.

Ensure that all datasets are correctly mounted before moving to the next step.

1. clone another encrypted dataset as raw data:

```
# zfs send -Rw zroot/encr@migration | zfs recv -F extzfs/encr
```

This will recursively clone all the datasets under a new encrypted dataset called `extzfs/encr/encr`. The new encryption root will have the same key as the source dataset, so you will be able to unlock it with the same passphrase. All properties and mountpoints will also be kept.

Given that all properties have been preserved, it may be enough to run

```
# zfs mount -la
```

to unlock and mount all new datasets. If that doesn't result in correctly mounted datasets, ensure that all properties (including mountpoints) have been correctly preserved.

3.3.5. Migrating filesystems: wrapping up

Regardless of the method you've picked, you should now have a working system on the new disk. Chroot into it as described in section 3.2., and then proceed to the next step.

3.4. Configuring the base system

Regardless of whatever path you took, you should now be in a working Arch Linux chroot.

3.4.1. Basic configuration

Most of the pre-boot configuration steps now are basically the same as a normal Arch Linux install:

```
[root@chroot /]# nvim /etc/pacman.d/mirrorlist # enable your favourite mirrors
[root@chroot /]# nvim /etc/locale.gen         # enable your favourite locales (e.g. en_US.UTF-8)
[root@chroot /]# locale-gen                   # generate the locales configured above
```

The next step is to populate the `/etc/fstab` file with the correct entries for all your partitions. Remember to use PARTUUIDs or plain UUIDs, and never rely on disk and partition names (except for `/dev/mapper` device files). The contents of `/etc/fstab` will vary depending on the filesystem you've picked. Remember that the `initrd` will be the one to unlock the LUKS container, so you don't need to specify it in `/etc/crypttab`.

- `/etc/fstab` for ext4/XFS/F2FS with LUKS:

```
# it is not strictly necessary to also include the root partition, but it's a good idea in cas
/dev/mapper/ExtLUKS          /                ext4      defaults    0 1
PARTUUID=4a0eab50-7dfc-4dcb-98a6-ad954d344ad7 /boot/efi        vfat      defaults    0 2
```

- `/etc/fstab` for Btrfs with LUKS:

```
/dev/mapper/ExtLUKS          /                btrfs    defaults,subvol=@,compres
/dev/mapper/ExtLUKS          /home            btrfs    defaults,subvol=@home,com
PARTUUID=4a0eab50-7dfc-4dcb-98a6-ad954d344ad7 /boot/efi        vfat      defaults
```

- With ZFS, all datasets mountpoints are managed via the filesystem itself. `/etc/fstab` will only contain the ESP (unless you have created legacy mountpoints):

```
PARTUUID=4a0eab50-7dfc-4dcb-98a6-ad954d344ad7 /boot/efi        vfat      defaults    0 2
```


Then, set a password for root:

```
[root@chroot /]# passwd
New password:
Retype new password:
passwd: password updated successfully
```

and create a standard user. Remember to mount `/home` first if you are using a separate partition or subvolume!

```
[root@chroot /]# mount /home # if using a separate partition or subvolume, not needed with ZFS
[root@chroot /]# useradd -m marco # this is my name
[root@chroot /]# passwd marco
New password:
Retype new password:
passwd: password updated successfully
```

Before moving to the next step, install all packages required for connectivity, or you may be unable to connect to the internet after you boot up the system.

For simplicity, I'll just install `NetworkManager` :

```
[root@chroot /]# pacman -S networkmanager
[...]
```

As the last step before moving to the next point, remember to configure the correct console layout in `/etc/vconsole.conf` , or you will have a hard time typing your password at boot time (the file will be copied to the *initrd*):

```
[root@chroot /]# cat > /etc/vconsole.conf <<'EOF'
KEYMAP=us
EOF
```

3.4.2. Configuring the kernel

Configuring the system for booting on multiple systems is easier than it sounds, thanks to how good Linux and the graphical stack has become at automatically configuring itself depending on the hardware.

In the `chroot` , run the following preliminary steps:

1. (optional) First, install ZFS (if you are using it); if using the LTS kernel, I recommend using `zfs-dkms` , while for a more up-to-date kernel a “fixed” build such as `zfs-linux` is probably safer.
2. In order to support systems with an NVIDIA GPU, install the Nvidia driver (`nvidia` or `nvidia-lts` , depending on what you’ve chosen) ¹⁴.
3. Install the microcode for *both* Intel and AMD CPUs (`intel-ucode` and `amd-ucode` respectively). Only the correct one will be loaded at boot time.

With the kernel and all necessary modules installed, we can now generate a bootable image.

For this step I’ve decided to use UKI, which is a novel approach to initramfs that simplifies the process a lot, by merging together kernel and initrd into a single bootable file. This is not strictly necessary, but it allows us to avoid messing the ESP with the contents of `/boot` : only UKIs and the (optional) bootloader will need to reside on it.

UKIs can be generated with several initramfs-generating tools, such as `dracut` and `mkinitcpio` . After a somewhat long stint with `dracut` , I’ve recently switched to `mkinitcpio` (Arch’s default) due to how simple it

is to configure and customize with custom hooks.

For a portable system, it's best to always boot using the `fallback` preset. The default preset generates a `initramfs` custom tailored to the current hardware, which may not work on other systems except the one that generated it. The `fallback` preset, on the other hand, generates a generic `initramfs` that contains by default the modules needed to boot on (almost) any system. The size difference may have been significant in the past, where disk space was small and expensive, but nowadays it's negligible. A UKI image generated with the `fallback` preset is around 110 MiB in size, which is enough to fit on our 300 MiB ESP.

First, we ought to create a file containing the command line arguments for the kernel.

The kernel command line is a set of arguments passed to the kernel at boot time, which can be used to configure how the kernel, the `initramfs` or `systemd` will behave. Under UEFI, these parameters are usually passed by a bootloader as boot arguments to the kernel when invoked from the ESP. UKI differs in this regard by directly embedding the command line in the image itself.

Create a file called `/etc/kernel/cmdline` with at least the following contents; feel free to add more parameters if you need them.

For LUKS:

```
rw nvidia-drm.modeset=1 cryptdevice=PARTUUID=5c97981e-4e4c-428e-8dcf-a82e2bc1ec0a:ExtLUKS root=/de
```



Omit the `rootflags` and `rootfstype` parameters if you are not using `Btrfs`.

For ZFS, try something akin to the following:

```
rw nvidia-drm.modeset=1 zfs=extzfs
```

which relies on automatic `bootfs` detection in order to find the root dataset.

After this, edit the `/etc/mkinitcpio.conf` to add any extra modules and hooks required by the new system.

You probably want to load `nvidia` KMS modules early, in order to avoid any issues when booting on systems with an NVIDIA discrete GPU. Notice that this may sometimes cause issues with buggy laptops with hybrid graphics, so remember this tradeoff in case you are incurring on this issue.

```
MODULES=(nvidia nvidia_drm nvidia_uvm nvidia_modeset)
```

The hooks you pick and the order in which they are run are crucial for a working system. For instance, if you are using encrypted ZFS, this is a safe starting point:

```
HOOKS=(base udev autodetect modconf kms block keyboard keymap consolefont zfs filesystems fsck)
```

For LUKS,

```
HOOKS=(base udev autodetect modconf kms keyboard keymap consolefont block encrypt filesystems fsck)
```



Notice how the `keyboard` and `keymap` hooks have been specified before either the `zfs` or `encrypt` hooks. This ensures that the keyboard and keymap are correctly configured before reaching the root encryption password prompt.

Before triggering the generation of our image, we must enable UKI support in the fallback preset (and disable the default one).

Edit `/etc/mkinitcpio.d/linux-lts.preset` as follows:

```
# mkinitcpio preset file for the 'linux-lts' package

#ALL_config="/etc/mkinitcpio.conf"
ALL_kver="/boot/vmlinuz-linux-lts"
ALL_microcode=(/boot/*-ucode.img)

PRESETS=('fallback')

#default_config="/etc/mkinitcpio.conf"
#default_image="/boot/initramfs-linux-lts.img"
#default_uki="/boot/efi/EFI/Linux/arch-linux-lts.efi"
#default_options="--splash /usr/share/systemd/bootctl/splash-arch.bmp"

#fallback_config="/etc/mkinitcpio.conf"
#fallback_image="/boot/initramfs-linux-lts-fallback.img"
fallback_uki="/boot/efi/EFI/BOOT/Bootx64.efi"
fallback_options="-S autodetect"
```

In the preset above, I have completely disabled out the `default` preset by removing it from `PRESETS` and commenting all of its entries. Under `fallback`, I only kept the `uki` and `options` entries, in order to avoid generating an initramfs image that we have no use for.

Run `mkinitcpio -p linux-lts` to finally generate the UKI under `/boot/efi/EFI/BOOT/Bootx64.efi`, which is the custom path I set `fallback_uki` to. This is the location conventionally associated with the UEFI Fallback bootloader, which will make the external drive bootable on any UEFI system without the need of any configuration or bootloader, as long as booting from USB is allowed (and UEFI Secure Boot is off).

```
[root@chroot /]# mkdir -p /boot/efi/EFI/BOOT # create the target directory
[root@chroot /]# mkinitcpio -p linux-lts
[...]
```

Optionally, clean up `/boot` by removing the initramfs images previously generated by `pacman` when installing the kernel package. These are unnecessary when using UKIs, and will never be generated again with the modifications we made to the kernel preset:

```
# rm /boot/initramfs*.img
```

3.4.2. Installing a bootloader (optional)

In principle, the instructions above make having a bootloader at all somewhat redundant. With UEFI, you can also always tinker with command line arguments using the UEFI Shell, which can be either already installed on the machine you are booting on or copied in the ESP under `\EFI\Shellx64.efi`.

In case you want to install a bootloader, change the `fallback_uki` argument to a different path (i.e. `/boot/efi/EFI/Linux/arch-linux-lts.efi`) and then just follow [Arch Wiki's instructions on how to set up systemd-boot](#) (or `rEFInd`, or `GRUB`, or whatever you like).

If you opt for `systemd-boot`, ensure that `bootctl install` copies the bootloader to `\EFI\BOOT\Bootx64.efi`, or it will not get picked up by the UEFI firmware automatically.

3.5. Unmounting the filesystems

Before attempting to boot the system, remember to unmount all filesystems and close the LUKS container. After ensuring you followed all the steps above correctly, exit the chroot, and then:

```
[root@chroot /]# exit
$ sudo umount -l /tmp/mnt/{dev,sys,proc,run} # the '-l' flag prevents issues with busy mounts
```

If you used LUKS:

```
$ sudo umount -R /tmp/mnt
```

If you used ZFS, you also have to remember to export the pool - otherwise, the pool will still be in use next boot, and the initrd scripts won't be able to import it:

```
$ sudo zpool export extzfs
```

This command may sometimes fail with an error message similar to `cannot export 'extzfs': pool is busy`. This is usually caused by a process still using the pool, such as a shell with its current directory set to a directory inside the pool. If this happens, the fastest way to fix it is to reboot the system, import the pool (without necessarily unlocking any dataset), and then immediately export it. This will ensure that the pool is not in use and untie it from the current system's `hostid`.

4. Booting the system

If you've followed the instructions above, you should now have be able to boot onto the new system successfully, without any troubleshoot necessary.

You can either test the new system by booting from native hardware, or inside a virtual machine.

4.1. Setting up a VM

In order to spin up a VM, you need a working hypervisor. If you intend to run the VM on a Linux host, `Qemu` with `KVM` is an excellent choice.¹⁵

You can either use `Qemu` via `libvirt` and tools such as `virt-manager`, or use plain `QEMU` directly. The former tends to be way easier to setup, but more troublesome to troubleshoot; `libvirt` is unfortunately full of abstractions that make configuring `Qemu` harder than just invoking it with the right parameters. On the other hand, `libvirt` automatically handles unpleasant parts such as configuring network bridges and `dnsmasq`, which you are otherwise required to configure manually.

Regardless of what approach you prefer, you should install UEFI support for guests, which is usually provided in packages called `ovmf`, `edk2-ovmf`, or similar.

4.1.1. Using `libvirt`

If you are using `libvirt`, you can use `virt-manager` to create a new VM (or dabble with `virsh` and XML directly, if that's more to your liking). If you opt for this approach, remember to:

1. Select *the device*, and not partitions or `/dev/mapper` devices. The disk must be unmounted and no partitions should be in use. Pick "Import an image" and then select `/dev/disk/by-id/usb-XXX`, without `-partN`, via the "Browse local" button.
2. Select "Customize configuration before install", or you won't be able to enable UEFI support. In the configuration screen, in the `Overview` pane, select the "Firmware" tab and pick an x86-64 `OVMF_CODE.fd`. If you don't see any, check that you've installed all the correct packages.

3. (optional) If you wish, you may enable VirGL in order to have a smoother experience while using the VM. If you're interested, toggle the “OpenGL” option under the `Display Spice` device section. Also remember to disable the SPICE socket, by setting the `Listen type` for SPICE to `None`. Check that the adapter model is `Virtio`, and enable 3D acceleration.¹⁶

4.1.2. Using raw `qemu`

Using plain Qemu in place of `libvirt` is undoubtedly less convenient. It definitely requires more tinkering for networking (especially if you don't want to use SLIRP, which is slow and limited), with the advantage of being more versatile and not requiring setting up `libvirt` - which tends to be problematic on machines with complex firewall rules and network configurations.

First, make a copy of the default UEFI variables file:

```
$ cp /usr/share/ovmf/x64/OVMF_VARS.fd ext_vars.fd
```

Then, temporarily take ownership of the disk device, in order to avoid having to run `qemu` as root:

```
$ sudo chown $(id -u):$(id -g) /dev/disk/by-id/usb-Samsung_SSD_960_EVO_250G_012938001243-0:0
```

Finally, run Qemu with the following command line. In this case, I'll use SLIRP for simplicity, plus I will enable VirGL for a smoother experience:

```
$ qemu-system-x86_64 -enable-kvm -cpu host -m 8G -smp sockets=1,cpus=8,threads=1 -drive if=pflash,
```

4.2. Booting on bare hardware

The disk previously created *should* be capable of booting on potentially any UEFI-enabled x86-64 system, as long as booting from USB is allowed and Secure Boot is disabled.¹⁷

At machine startup, press the “Boot Menu” key for your system (usually F12 or F8, but it may vary considerably depending on the vendor) and select the external SSD. The disk may be referred to as “fallback bootloader” - this is normal, given that we've placed the UKI image at the fallback bootloader location.

4.3. First boot

If you did everything right in the last few steps, the boot process should stop at a password prompt from either `cryptsetup` (LUKS) or `zpool` (ZFS).

Insert the password and press enter. If everything went well, you should now be greeted by a login prompt.

Login as root, and proceed with the last missing configuration steps:

1. If you are running on ZFS, you'll notice that `/home` and `/root` are not mounted automatically. In order to fix this, immediately run

```
# systemctl enable zfs.target zfs-mount.server zfs-import.target
```

After doing this, **reboot the system** and check that the datasets are mounted correctly. You shouldn't need to enable `zfs-import-cache.service` or `zfs-import-scan.service` as they are unnecessary, given that we're booting from a single pool which is already imported.

1. Enable and start up the network manager of your choice you've installed previously, such as

```
NetworkManager : # systemctl enable --now NetworkManager
```

If you are using a wired connection with DHCP or IPv6 and no special configuration required, you should see any relevant IPs under `ip address`, and Internet should be working.

If you need special configurations, or you must use wireless connectivity, use `nmtui` to configure the network.

2. With a booted instance of `systemd`, you can now easily set up everything else you are missing, such as:

- a **hostname** with `hostnamectl set-hostname` ;
- a **timezone** with `timedatectl set-timezone` (you may need to adjust it depending on where you boot from);
- if you know as a fact you are always going to boot from systems with an RTC on localtime, set `timedatectl set-local-rtc 1` to avoid having to adjust the time every time you boot. Note that this is arguably one of the most annoying parts about a portable system; I recommend setting every machine you own to UTC and properly configuring Windows to use UTC instead.
- a different locale (generated via `locale-gen`), in order to change your system's language settings.

As an example:

- Use `localectl set-locale LANG=en_US.UTF-8` to set the default locale to `en_US.UTF-8`
- Use `localectl set-keymap de` to change the keyboard layout to German.

4.4. Installing a desktop environment

The most useful part about a portable system is to carry a workspace around, so you can work on your projects wherever you are.

In order to do this, you need to install some kind of desktop environment, which may range from minimal (`dwm` , `sway` , `fluxbox`) to a full fledged environment like Plasma, GNOME, XFCE, Mate, ...

Just remember that you are going to use this system on a variety of machines, so it's useful to avoid anything that requires an excessive amount of tinkering to function properly. For instance, if one or more of the systems you plan to target involve NVIDIA GPUs, you may find running Wayland more annoying than just sticking with X11.

4.4.1. Example: Installing KDE Plasma

I'm a big fan of KDE Plasma (even though I've been using GNOME recently, for a few reasons), so I'll use it as an example.

In general, all DEs require you to install a metapackage to pull in all the basic components (like the KF5 frameworks) and an (optional display manager), plus some or all the applications that are part of the DE.

If you plan on running X11, install the `xorg` package group, and then install `plasma` :

```
# pacman -S plasma plasma-wayland-session sddm kde-utilities
```

If you are using a display manager, enable it with `systemctl enable --now sddm`.

Otherwise, either configure your `.xinitrc` to start Plasma by appending

```
export DESKTOP_SESSION=plasma
exec startplasma-x11
```

and run `startx`.

If you prefer using Wayland, just straight out run `startplasma-wayland` instead.

5. Basic troubleshooting

If you followed all steps listed above, you *should* have a working portable system. Most troubleshooting steps after the initial booting should be identical to those of a normal Arch Linux system. Below you'll find a very basic list of a few common issues that may arise when attempting to boot the system on different machines.

5.1. Device not found or No pool to import during boot

If the `initrd` fails to find the root device (or the ZFS pool), it means that the `initrd` failed to correctly mount the correct drive. This it's often due to the following three reasons:

1. The `initrd` is missing the required drivers. The disk is not appearing under `/dev` because of this.

The `fallback` `initrd` is supposed to contain all the storage and USB drivers needed to boot on any system, but it's possible that some may be missing if your USB controller is either particularly exotic or particularly quirky (e.g. Intel Macs).

First, on the affected system, try to probe what drivers are in use for your USB controller. You can use `lspci -k` from a Linux system you can mount the external disk from:

```
$ lspci -k
[...]  
0a:00.3 USB controller: Advanced Micro Devices, Inc. [AMD] Family 17h (Models 00h-0fh) USB 3.0  
Subsystem: Gigabyte Technology Co., Ltd Family 17h (Models 00h-0fh) USB 3.0 Host Controll  
Kernel driver in use: xhci_hcd  
Kernel modules: xhci_pci  
[...]
```

Afterwards, add the relevant module(s) to the `MODULES` array in `/etc/mkinitcpio.conf`, and regenerate the `initrd`.

2. The kernel command line is incorrect. The `initrd` either has the wrong device set, or the kernel is not receiving the correct parameters.

This happens either due to a bad `root` or `zfs` line in `/etc/kernel/cmdline`, or because a bootloader or firmware are passing spurious arguments to the UKI.

Double check that the `root` or `zfs` line in `/etc/kernel/cmdline` is correct. Some bootloaders such as *rEFInd* support automatic discovery of bootable files on ESPs; it may also be that the bootloader is wrongly assuming the UKI is a EFISTUB-capable kernel image and passing incorrect flags instead.

In any case, ascertain that the kernel is actually receiving the correct parameters by running

```
# cat /proc/cmdline
```

from the `initrd` recovery shell.

If you are using ZFS and you only specified the target pool instead of the root dataset, remember to set `bootfs` correctly first.

3. **(ZFS only)** An incorrect cache file has been embedded in the `initrd`. The `initrd` is trying to use potentially incorrect pool data instead of scanning `/dev`.

The `zfs` hook embeds `/etc/zfs/zpool.cache` into the `initrd` during generation. While this is often useful to reduce boot times, especially with large multi-disk pools, it may cause issues if the cache file is stale or incorrect. Return back to the setup system, `chroot`, remove the cache file and regenerate the UKI. The

initrd should now attempt discovery the root pool via `zpool import -d /dev` instead of using the `cachefile` (or any `zfs_import_dir` you may have set via the kernel command line).

If none of the previous steps work, you may want to try to boot the system from a different machine to ensure there's not a problem in the setup itself.

5.2 The keyboard doesn't work properly at the password prompt

1. If the keyboard doesn't work when typing the encryption password, it's probably due to the keyboard hook not being run before the encryption hooks (whatever you are using). Ensure that `keyboard` is listed before `encrypt` or `zfs` in `/etc/mkinitcpio.conf`.
2. If the keyboard is working, but the password is not being accepted, it may be due to an incorrectly set keyboard layout. Ensure that `/etc/vconsole.conf` is set correctly, and that the `keymap` hook is being run before the encryption hooks.

5.3. The system boots, but the display is not working

This is rarely an issue with Intel or AMD GPUs, but it's pretty common with NVIDIA GPUs, especially on buggy laptops with Optimus hybrid graphics.

1. Remember to always enable KMS modules early, in order to avoid any issues when booting on systems with an NVIDIA discrete GPU. Append `nvidia-drm.modeset=1` to the kernel command line, and add the `kms` hook right after `modconf` in `/etc/mkinitcpio.conf`. This should force whatever KMS driver you are using to load early in the boot process, which should provide a working display as soon as the `initrd` is loaded.

Note that with NVIDIA the framebuffer resolution is often not increased automatically, which may lead to a poor CLI experience. This is a common issue that unfortunately tends only to affect NVIDIA users.

2. Add `nvidia nvidia_modeset nvidia_uvm nvidia_drm` to the `MODULES` array in `/etc/mkinitcpio.conf`. This will ensure that the NVIDIA driver is always loaded early in the boot process. The module will be ignored and unloaded if not needed on the system currently in use.
3. Do not use any legacy kernel option such as `video=` or `vga=`. There are lots of old guides still suggesting to use them, but they are not compatible with KMS and should not be used anymore.

5.4. It's impossible to log in via a display manager, or logging from a `tty` complains that the user directory is missing

This is an issue almost always caused by `/home` not being mounted correctly. Either check that `/home` is correctly configured in `/etc/fstab`, or that `zfs-mount` is enabled and running alongside the `zfs` target.

6. Conclusion

This post is a very basic guide on how to set up Arch Linux on a portable SSD, which I think feels less like a manual and more like my personal notes.

This is intentional: while nothing in this guide is unique (everything can be found in the Arch Wiki, in forums or in other blogs), I felt that it was worth it gathering some of my personal experience in a single place, hopefully with the intent of it being useful to someone else besides myself.

I suspect that after installing Linux (and Arch in particular) an infinite number of times, I grew a bit desensitised to how tricky and error-prone the process can be, especially for newcomers and people who are not accustomed to system administration and troubleshooting. Hopefully, the knowledge written in this article will be a good starting point for anyone who wants to try out Arch Linux, and maybe also get a cool portable system out of it.

Thanks a lot for reading, and as always feel free to contact me if you find anything incorrect, imprecise or hard to understand.

1. and Wi-Fi. Wi-Fi was a PITA too, and don't get me started on "retches" USB ADSL modems with Windows-only drivers on mini CDs. ↩
2. YMMV. Some devices (e.g. Macs) are notoriously picky about booting from USB drives, but that's not our system's fault. ↩
3. E.g. if you drop it, there's a non-zero chance the USB connector and/or the logic board will break. USB enclosures are often very cheap compared to SSDs, so using them is the smarter choice in the long run. ↩
4. ARM would be interesting too, if it wasn't for the fact that there's nothing akin to PC standards for ARM devices, and even today in 2023 it's still a hodgepodge of ad-hoc systems and clunky firmware. The fact that lots of ARM devices are also severely locked down doesn't help, either. ↩
5. SSDs and HDDs are complex systems and may fail in several ways, which may lead to situations where the data on the disk is still readable using specialised tools, but cannot be accessed, deleted or overwritten using a normal computer (i.e. if the SSD controller fails). Properly encrypted disks are fundamentally random data, and as long as the encryption scheme is secure and the password is strong, you can chuck a broken disk in the trash without losing sleep over it. ↩
6. Using ZFS is also a lot of fun IMHO. ↩
7. If you suspect you may be a potential target for evil maid attacks, you should probably refrain from using a portable install altogether. ↩
8. A small warning: compared to similar tools, parted writes changes to the disk immediately, so always triple-check what you're doing before hitting enter. I recommend sticking to `gdisk` due to its better support for automatic alignment of partitions. ↩
9. `gparted` also supports advanced features such as resizing filesystems, which is very handy when you don't want to use the whole disk for the installation. It is also possible to perform such tasks from the command line, but it is in general more complex and error-prone. ↩
10. Linux has no "absolute" naming policy for raw block devices. In particular, USB mass storage devices are enumerated alongside the SCSI and SATA devices, so it's not uncommon for a USB disk to suddenly become `sda` after a reboot. ↩
11. Once I've lost a ZFS pool due to a bug in a Git pre-alpha release of OpenZFS. That day, I learnt that running an OS from a pre-alpha filesystem driver is not a hallmark of good judgement. ↩
12. If you compile pacman and/or use an Arch chroot, it's absolutely doable from any distro, really, as long as its kernel is new enough to run Arch-distributed binaries. See section 3.2.2. to learn how to do this. ↩
13. Notice that I'm using `perl-rename` in place of `rename`, because I honestly think that plain `rename` is just outright terrible. `perl-rename` is a Perl script that can be installed separately (on Arch is in the `perl-rename` package) and it's just better than util-linux' `rename` utility in every measurable way. ↩
14. I don't recommend using `nvidia-open` or Nouveau as of the time of writing (October '23), due to the immature state of the first is and the utter incompleteness the latter. The closed source `nvidia` driver is still the best choice for NVIDIA GPUs, even if it sucks due to how "third-party" it feels (its non-Mesa userland is particularly annoying). ↩
15. On Windows, you can also consider using Hyper-V, which has also the advantage of being already included in Windows and supports using real device drives as virtual disks. ↩
16. This feature is known to be buggy under the closed-source NVIDIA driver, so beware. ↩
17. Using Secure Boot with an external disk you plan on carrying around is very troublesome for a variety of reasons - first and foremost that you'd either have to enroll your personal keys on every system you plan on booting from, or plan on using Microsoft's keys, which means fighting with `MokList.S`, `PreLoader.efi`, and going through a lot of pain for very dubious benefits. ↩

Related Posts

Unicode is harder than you think 23 Jul 2023

Cross compiling made easy, using Clang and LLVM 09 Feb 2021

NAT66: The good, the bad, the ugly 20 Jan 2018